Neuro Copilot

System Design Submission | Team: Benora| Members: Mohammed Faroz Khan ,Vinay Kumar ,Mohammed Aaquib , Ansh Patel

Problem

Fragmented data: customer data sits in separate loan, card, transaction and collections systems.

Wasted call time: agents spend it gathering context.

Missed promises to pay (PTP) and decisions that vary from agent to agent.

Use case

Agentic Collections Copilot (plans steps and calls tools, not just chat).

Builds one Golden Customer 360 (C360) per customer.

Gives an explainable honor score (0-100: likelihood the customer keeps paying) and recommends the next best action.

Who uses it

Bank collections agents, supervisors and hardship specialists.

The agent verifies every recommendation.

The copilot is decision support, never an autonomous decision-maker.

Business value

METRIC WE IMPROVE

PTP adherence

= promises kept on time / promises made

Supporting: days past due (DPD), roll rate (share of accounts slipping into a later lateness bucket), cost per recovery, agent minutes spent searching for context per call.

HOW WE MEASURE

A/B test: copilot group vs manual group, matched on product and DPD bucket.

Duration: 4-6 weeks, compared against a 90-day baseline.

Evidence: confidence intervals show how sure we are the gain is real.

Hackathon: we simulate the test on synthetic (fake but realistic) data.

Architecture: four layers, AI and human review

DATA SOURCES	Loan system	Card system	Transactions	Collections notes + call audio
LAYER 1 Customer 360 data product: one golden record per customer	**Main database (PostgreSQL)** Source of truth. Append-only, form-based entry: history is never overwritten.	**Kafka (live) + Spark (nightly)** Kafka: events within 5 min. Spark: nightly balances, loan schedules, score refresh. Airflow schedules jobs.	AI **Identity matching** Exact: phone, email. Fuzzy: names, addresses (Splink). Doubtful pairs go to a data steward. Output: golden_customer_id.	**Golden C360 + Honor Score tables** Buckets: loan, card, collections, scores. score_reasons table. Lineage on every field. Data contract enforced.
LAYER 2 Questions to answers (Insight and NLP), read-only	AI **Access check + Guardrail + Router** Role check, then REFUSE PII, out-of-scope and prompt-injection requests. Claude LLM routes the rest.	AI **Text-to-SQL + pandas** Exact numbers. SQL is shown. Read-only, one customer only. Data dictionary in prompt.	AI **RAG (pgvector)** Policies, notes, transcripts. Cites retrieved chunk IDs only. Retrieved text treated as data.	AI **Validator + merge** Every number checked against tool output. Data timestamp shown. Fails: answer blocked.
LAYER 3 Feature Store (Feast): same values for training and live scoring	**Feature registry** Each feature defined ONCE, versioned. One definition feeds both stores.	**Offline store (Parquet + Spark)** History for training. Point-in-time joins stop future data leaking in.	**Online store (Redis)** Latest values, refreshed by Kafka events in seconds, read at live scoring.	AI **Text + voice features** Whisper speech-to-text; hardship terms, sentiment, promise language (LLM extraction).
LAYER 4 Models and Decisioning	AI **Risk + early-warning (XGBoost)** Honor score 0-100. Missed-promise alert 2-3 days ahead. SHAP gives reason codes. What-if re-scoring.	**Rules engine (decides)** Eligibility: extension, fee waiver, interest cut, restructure. Versioned policy table with effective dates.	AI **LLM explanation card (explains)** Timestamp, key facts, options, policy clause + date, reason, approval needed, confidence, missing data.	AI **Routing** Early-stage, low risk: approved SMS/email templates. Complex, high-risk or hardship flag: human agent.
HUMAN REVIEW before anything reaches the customer	**Collections agent** Verifies and edits every recommendation. Confirms the post-call note.	**Supervisor** Approves any concession or change to the customer's contract.	**Hardship specialist** Takes over when hardship flag fires. Automated contact stops.	**Data steward** Reviews doubtful ID matches and overdue policy reviews.

Feedback loop: post-call note and decision are confirmed by a human, then published to Kafka and the audit trail, feeding Layer 1 again.

AI used here (purple tag) Human reviews here (orange) Tool, data or rule (no AI)

Layers 1 and 2: from raw data to trusted answers

Layer 1 Build the C360, match customer IDs

Source of truth: PostgreSQL main database, append-only with form-based entry, so full history is kept.

Pipelines: Kafka carries real-time events (payment, promise, call end) within seconds. Spark runs nightly balances, loan schedules and honor score recompute. Airflow orchestrates.

Identity resolution: Step 1, exact match on normalised phone and email. Step 2, fuzzy match on names and addresses with Splink. Score above 0.95: auto-merge. 0.80 to 0.95: data steward decides. Below 0.80: kept separate. A mapping table (source system, source ID, golden ID) keeps every merge reversible.

Customer buckets: loan, card, collections, scores. Core fields: golden_customer_id, total_balance_due, dpd_max_current, ptp_broken_count_90d, hardship_signal.

Honor score tables: honor_score plus score_reasons (reason code, feature, impact), joined on customer ID so any score can be explained.

Lineage: every field carries its source system and last-updated time; the data contract sets owner, quality rules and refresh.

Layer 2 Questions to answers, and refusals

Flow: access check, guardrail, router, tools, validator, answer.

Router: Claude LLM decides whether a question needs exact data, policy or notes text, or both.

Text-to-SQL + pandas: exact numbers; SQL shown to the agent. Read-only account, views for the opened customer only, data dictionary in the prompt.

RAG (pgvector): policies, agent notes, call transcripts. Cites retrieved chunk IDs only; each ID is verified before display.

Validator: every number must match tool output or the answer is blocked. Data timestamp on every answer.

How we refuse: the guardrail blocks (a) PII of other customers or raw ID documents, (b) questions unrelated to this account, such as legal advice, (c) requests to change records or approve concessions, (d) prompt injection hidden in notes. Uses a rule list, a scope check and a small classifier. The refusal says why and offers the manual path; all refusals are logged and tested on a red-team set.

Layers 3 and 4: features, models and decisions

Layer 3 Feature Store (Feast)

Source	Features we build	How
Structured (loan, card, transactions, collections)	ptp_broken_count_90d, dpd_max_current, days_since_last_payment, payment-to-due ratio (3 months), card utilisation, minimum-payment-only streak, balance trend (30/60/90 days), contact attempts, response rate by channel	Spark nightly for history; Kafka events update latest values
Text (notes, chat, emails)	hardship_signal (e.g. "lost job"), dispute mentioned, promise wording ("I will pay Friday" vs "I will try"), sentiment	Keyword list + LLM extraction into fixed fields; claims stored as "customer states"
Voice (call recordings)	Same text features, plus talk-time ratio and call outcome	Whisper speech-to-text, then the text pipeline

Same values for training and live scoring

One definition: each feature is defined once in the versioned feature registry and fills two stores.

Offline store: Parquet history read by Spark, used for training with point-in-time joins (each row sees only what was known that date).

Online store: Redis holds the latest values, read at live scoring.

Skew check: a nightly job compares online vs offline values and alerts on any mismatch.

Layer 4 Models and Decisioning

Use case: next best action for collections: which concession, if any, to offer, and when to contact.

Models: XGBoost default-risk classifier gives the honor score (0-100). A second early-warning model predicts a missed promise 2-3 days before it is due.

Explained by: SHAP writes the top reasons into score_reasons. A what-if view re-scores the customer if they pay amount X.

Rules engine decides: fixed if-then, versioned policies with effective dates decide eligibility for extension, fee waiver, interest reduction or restructuring. The LLM only explains.

Recommendation card: data timestamp, key facts, options, policy clause with date, reason, supervisor approval needed, confidence and missing data.

Routing: early-stage, low-risk accounts get compliance-approved SMS/email reminders (no concessions). Complex or high-risk cases go to a human agent.

Where a human reviews: the agent verifies every recommendation; a supervisor approves any concession or contract change; a hardship specialist takes over when the hardship flag fires; after the call the human confirms the drafted note and the event goes to Kafka. Manual mode is always available.

AI usage, scalability and governance

AI usage: where and why

Fuzzy ID matching (Splink): names and addresses are messy; exact match misses duplicates.

Routing, Text-to-SQL, RAG (Claude): agents ask in plain English.

Transcription, hardship, sentiment (Whisper + LLM): hardship is often said, not typed.

Honor score, early warning (XGBoost + SHAP): learns history; each score explainable.

Explanation card, pre/post-call summary (Claude): saves reading and typing; human confirms.

Policy review alerts (LLM + scheduler): flags policies past review date.

AI NOT used for: eligibility (rules), approvals (humans), numbers (read from database, then validated).

Risks to fixes: wrong numbers: validator. Fake citations: verified chunk IDs. Stale data: timestamp. Misread columns: data dictionary. Latency: parallel calls. Error rate: measured on a test set and reported.

Scalability: batch vs real time

BATCH (nightly, Spark + Airflow)

Balance reconciliation, loan schedules, full honor score recompute, full identity-matching pass, offline feature backfill, fairness and executive dashboards, policy review alerts.

REAL TIME (Kafka, seconds to minutes)

Payment, PTP made/broken and hardship events within 5 minutes; online feature updates; copilot answers (target under 5 seconds); early-warning refresh on key events.

Scaling: Kafka partitioned by golden_customer_id; Spark adds machines; Redis serves features in milliseconds; LLM tool calls run in parallel.

Production: copilot runs as a separate read-only analytics layer, never slowing or changing core banking.

Governance

Access control: role-based. Agents see only assigned customers; every lookup is logged.

Human review: any concession or change to a customer's standing or contract needs human approval. Complex, high-risk and hardship cases always go to a person.

No protected attributes: age, gender, race, religion, marital status, disability and nationality are excluded from the score. Proxies (e.g. postcode) are tested. A fairness dashboard uses separately held data never fed to the model.

Hardship flags: terms like "medical emergency" or "lost job" stop automated collections immediately and escalate to a hardship specialist.

Trust and safety: customer claims stored as "customer states", never verified fact. Policies versioned with effective and review dates. Retrieved text is data, never instructions. Audit trail records every suggestion, choice and outcome.

Data contract (draft v0.1) and 36-hour build plan

golden_customer_360

Owner: Head of Collections Data*. Steward: Data Engineering lead.

Purpose: one trusted record per customer for collections prioritisation, copilot and early warning.

Sources: loan, card, transactions, collections notes, call transcripts.

Refresh: live events within 5 min via Kafka; balances, schedules, honor score nightly by 02:00; policies on every change.

Access: role-based; all reads logged. Retention: 7 years*.

Field	Type	PII
golden_customer_id	string (UUID), PK	No
total_balance_due	decimal(12,2)	No
dpd_max_current	integer	No
ptp_broken_count_90d	integer	No
hardship_signal	boolean	No
honor_score	integer 0-100	No
score_reasons	linked table	No
source_systems	array of string	No
last_updated_at	timestamp (UTC)	No
contact_phone, contact_email	string, masked	Yes

** assumption to confirm with the bank*

Quality rules (block publishing)

1. golden_customer_id unique, never null: 100%
2. total_balance_due reconciles to loan + card sources: within $0.01
3. Live events reach C360: within 5 minutes
4. honor_score 0-100 and not null for active accounts; dpd_max_current at least 0: 100%
5. Each source record maps to exactly one golden ID: 100%
6. No protected-attribute columns in score tables or feature store: 0 columns
7. Every field has source and last_updated_at: 100%

Allowed: collections prioritisation; human-reviewed copilot recommendations; early-warning alerts; internal dashboards (PTP adherence, roll rates, cost per recovery); fairness monitoring.

Not allowed: marketing or cross-selling; credit-limit or lending decisions; automated adverse action without human review; protected attributes; sharing outside the bank; roles without a collections need.

Build plan: 36 hours

0-4h Synthetic data (Faker): ~2,000 customers across 4 systems with deliberate duplicates; PostgreSQL set up.

4-10h Layer 1: identity matching, C360 tables, Kafka demo topic, one Spark nightly job.

10-16h Layer 3: Feast feature store; XGBoost honor score; SHAP reasons into score_reasons.

16-24h Layer 2: Text-to-SQL, RAG over ~10 sample policies, guardrail, validator.

24-30h Layer 4: rules engine, explanation card, hardship flag, pre-call brief in Streamlit.

30-34h Post-call summary, audit log, dashboards; accuracy test on ~50 sample questions.

34-36h Demo script, rehearsal, README. If behind, cut the early-warning model first.

EXPLANATION LINK

neuropilot-sand.vercel.app